

**SRS**

# Software Requirements Specification (SRS)

## TRCAC Blockchain Voting System (BVS)

### Degree College Module

**Document Version:** 1.0

**Prepared By:** Student Project Team

**Institution:** TRCAC (Thakur Ramnarayan College of Arts & Commerce)

**Date:** July 2026

**Status:** Final Draft

## Table of Contents

1. Introduction
2. Overall Description
3. User Roles & Characteristics
4. Functional Requirements
5. Non-Functional Requirements
6. System Constraints
7. External Interface Requirements
8. Data Requirements

## 1. Introduction

### 1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the **TRCAC Blockchain Voting System (BVS) — Degree College Module**. It is intended as a reference for the student development team, instructors, and any stakeholders involved in reviewing, testing, or extending the system.

## 1.2 Scope

The **TRCAC BVS** is a web-based platform designed to streamline secure and tamper-evident election activities at Thakur Ramnarayan College of Arts & Commerce. This document covers only the **Degree College** portion of the system, which manages the following degree programs:

| Program                          | Classes                   |
|----------------------------------|---------------------------|
| B.Sc (Computer Science)          | FYBScCS, SYBScCS, TYBScCS |
| B.Sc (Information Technology)    | FYBScIT, SYBScIT, TYBScIT |
| Bachelor of Arts                 | FYBA, SYBA, TYBA          |
| BAMMC                            | FYBAMMC, SYBAMMC, TYBAMMC |
| Bachelor of Management Studies   | FYBMS, SYBMS, TYBMS       |
| Bachelor of Commerce             | FYBCom, SYBCom, TYBCom    |
| Bachelor of Accounting & Finance | FYBAF, SYBAF, TYBAF       |

The system has three user roles: **Admin**, **Voter (Student)**, and **Observer (Faculty/Public)**, each with distinct dashboards and privileges.

## 1.3 Definitions & Acronyms

| Term         | Definition                                             |
|--------------|--------------------------------------------------------|
| BVS          | Blockchain Voting System                               |
| TRCAC        | Thakur Ramnarayan College of Arts & Commerce           |
| DC           | Degree College                                         |
| FY / SY / TY | First Year / Second Year / Third Year                  |
| JWT          | JSON Web Token — used for session authentication       |
| API          | Application Programming Interface                      |
| CRUD         | Create, Read, Update, Delete                           |
| PoW          | Proof-of-Work mining consensus algorithm               |
| ECDSA        | Elliptic Curve Digital Signature Algorithm (SECP256R1) |

## 1.4 References

- Project Codebase: trcac-blockchain-voting (GitHub Repository)
- DC Subject and Class List: DC class and candidate list.xlsx
- Security Documentation: SECURITY\_SPECS.md
- Architecture Documentation: SYSTEM\_ARCHITECTURE.md

## 1.5 Overview

The BVS is divided into two main logical layers:

9. **Frontend** — A React + Vite Single Page Application (SPA) deployed on Vercel
10. **Backend** — A FastAPI (Python 3.12) REST API connected to database and Blockchain Engine

## 2. Overall Description

### 2.1 Product Perspective

The TRCAC BVS Degree College module is a standalone web application that replaces manual, paper-based academic voting processes. The system is intended to be used by faculty members, students, and administrators of the degree college section of TRCAC.

The system integrates with:

- **SQLite / PostgreSQL** — cloud database
- **Custom Python Blockchain** — decentralized SHA-256 ledger
- **FastAPI WebSockets** — real-time vote updates

## 2.2 Product Functions (High-Level)

The following high-level capabilities are supported for the Degree College module:

- 11. User Account Management — Admin creates student and faculty accounts; role-based login
- 12. Election Management — Admin publishes elections; students and faculty view elections
- 13. Candidate Management — Admin manages candidate nominations per degree, year, and class
- 14. 1-Click Encrypted Voting — Vote choices encrypted via AES-256 GCM before ledger submission
- 15. Anti-Double Voting Enforcement — Single vote per student strictly enforced at database and blockchain layer
- 16. Digital Signature Signing — Transactions signed using single-use voter ECDSA key pairs
- 17. Proof-of-Work Block Mining — Mined blocks aggregate queued transactions with SHA-256 target difficulty
- 18. Bulk Enrollment — Admin bulk-enrolls students via CSV upload
- 19. Audit Log Tracking — System logs logins, vote events, and admin activities
- 20. Blockchain Explorer — Public block browser, transaction search, and 1-click chain audit
- 21. Settings / Password Management — Users can change their own passwords

## 2.3 User Classes

| User Class            | Description                                                                                         |
|-----------------------|-----------------------------------------------------------------------------------------------------|
| Admin                 | Full control of the system. Creates accounts, manages elections, candidates, and bulk data.         |
| DC Student (Voter)    | Casts votes, generates QR receipts, views active elections, and verifies ledger hashes.             |
| DC Faculty (Observer) | Monitors live election results, inspects block explorer, verifies chain, and views AI fraud alerts. |

## 2.4 Operating Environment

- **Client Side:** Modern web browser (Chrome, Firefox, Edge, Safari)
- **Server Side:** Python 3.12+, FastAPI 0.111+, Uvicorn 0.30+
- **Database:** SQLite (local) / PostgreSQL (cloud-hosted)
- **Frontend Build Tool:** Vite 5.x / 7.x
- **Deployment:** Frontend -> Vercel; Backend -> Railway / Render (port 8000)

## 2.5 Design and Implementation Constraints

- Frontend is built with **React 19** and **Tailwind CSS 3**
- Backend uses Python 3.12 with FastAPI REST framework
- JWT tokens expire in **24 hours** for security

- File uploads are limited to **10 MB** per file
- Allowed file types for bulk voter import: CSV
- Rate limiting is applied to all API endpoints
- CORS is strictly configured to allow only the registered frontend URL

## 2.6 Assumptions and Dependencies

- Users have stable internet connectivity.
- Every student possesses a valid institutional email.
- Database service remains available.
- JWT authentication service is operational.

## 3. User Roles & Characteristics

### 3.1 Admin

- **Technical proficiency:** Moderate; expected to use a web-based admin dashboard
- **Access level:** Full system access
- **Typical tasks:** Create voter/faculty accounts, manage elections, bulk enrollment
- **Authentication:** Stored in User table (role = 'admin')

### 3.2 Degree College Faculty (DC Faculty / Observer)

- **Technical proficiency:** Moderate
- **Access level:** Scoped to monitoring live standings and transparency metrics
- **Typical tasks:** View results, audit blockchain ledger, inspect AI fraud alerts
- **Authentication:** Stored in User table (role = 'observer')

### 3.3 Degree College Student (DC Student / Voter)

- **Technical proficiency:** Basic to moderate
- **Access level:** Scoped to their own class, year, and active election ballot
- **Typical tasks:** Cast encrypted vote, download QR receipt, verify receipt hash
- **Authentication:** Stored in User table (role = 'voter')

| Actor              | Use Cases                                                         |
|--------------------|-------------------------------------------------------------------|
| Admin              | Manage Users, Elections, Candidates, Bulk CSV Import              |
| Faculty / Observer | Live Standings, Audit Ledger, Chain Verification, AI Fraud Alerts |
| Student / Voter    | Cast Encrypted Vote, View Candidates, QR Receipt, Verify Hash     |

## 4. Functional Requirements

### 4.1 Authentication & Authorization Module

#### FR-AUTH-01: User Login

- **Description:** The system shall allow admin, faculty, and students to log in using email/username and password.
- **Input:** Email/Username, Password
- **Process:** System searches user record and verifies password using bcrypt. Returns a JWT token.
- **Output:** JWT access token + user profile data
- **Token Expiry:** 24 hours
- **Error Cases:** Invalid credentials -> HTTP 400

#### FR-AUTH-02: Admin User Creation

- **Description:** Only authenticated admins can create new user accounts.
- **Input:** Full name, email, password, role (student/faculty), degree, year, roll number
- **Process:** Admin-authenticated endpoint creates record in User table
- **Validation:** Roll number required for students; valid email required

#### FR-AUTH-03: Logout

- **Description:** Users can log out, clearing local storage JWT token.
- **Process:** Token removed from client storage; subsequent protected requests rejected.

#### FR-AUTH-04: Token Verification

- **Description:** Protected routes verify the Bearer token before proceeding.
- **Process:** JWT decoded -> user looked up by ID -> user attached to req.user

#### FR-AUTH-05: Password Change

- **Description:** Students and Faculty can change their own passwords.
- **Input:** Current password, new password (min 6 characters)
- **Process:** Verify current password -> bcrypt hash new password -> save

## 4.2 Admin Dashboard Module

### FR-ADMIN-01: Dashboard Overview

- **Description:** Admin home page displays aggregated stats:
  - Total Elections
  - Active Elections
  - Total Registered Voters
  - Total Blockchain Votes Mined
  - Blockchain Health Status
- **Data Source:** GET /api/v1/admin/metrics

### FR-ADMIN-02: Election Management

- **Description:** Admin can create, edit, activate, schedule, and close elections.
- **Operations:** List all elections; create new election; update status
- **Route:** POST /api/v1/elections, PUT /api/v1/elections/:id/status

### FR-ADMIN-03: Candidate Management

- **Description:** Admin can register candidates for an election with full metadata.
- **Fields:** Candidate Name, Party / Organization, Manifesto Description, Avatar URL
- **Route:** POST /api/v1/elections/:id/candidates

### FR-ADMIN-04: Bulk Student Enrollment

- **Description:** Admin can upload a CSV file to bulk-create student voter accounts.
- **CSV Fields:** Full Name, Email, Password, Roll No, Degree, Year
- **Route:** POST /api/v1/admin/import-voters-csv

### FR-ADMIN-05: Immutable Audit Logs

- **Description:** Admin can view and download immutable audit logs of all user activities.
- **Route:** GET /api/v1/admin/audit-logs

## 4.3 Student Dashboard Module (Degree College)

### FR-STUDENT-01: Student Home / Active Elections

- **Description:** Student home page displays greeting, active elections list, and candidate profiles.
- **Route:** GET /api/v1/elections

### FR-STUDENT-02: 1-Click Encrypted Voting

- **Description:** Student selects candidate and casts vote encrypted with AES-256 GCM and signed with ECDSA key pair.
- **Route:** POST /api/v1/votes/cast

### FR-STUDENT-03: Anti-Double Voting

- **Description:** System blocks duplicate vote attempts for the same election and logs violation to audit trail.

### FR-STUDENT-04: Downloadable QR Receipt

- **Description:** Generates printable QR receipt containing receipt hash, block index, and transaction hash.
- **Route:** GET /api/v1/votes/my-receipts

### FR-STUDENT-05: Receipt Verification

- **Description:** Students can verify any receipt hash against the public blockchain ledger.
- **Route:** GET /api/v1/votes/verify-receipt/:hash

## 4.4 Custom Blockchain & AI Module

### FR-BLOCKCHAIN-01: Proof-of-Work Block Mining

- **Description:** Automatically packages queued transactions into blocks meeting SHA-256 difficulty targets.

### FR-BLOCKCHAIN-02: Merkle Tree Generation

- **Description:** Computes cryptographic binary Merkle Root hash from transaction hashes.

### FR-BLOCKCHAIN-03: Ledger Explorer & Tamper Audit

- **Description:** Public explorer endpoint allowing block browsing and 1-click full chain validation.
- **Route:** GET /api/v1/blockchain/blocks, GET /api/v1/blockchain/verify-chain

### FR-AI-01: AI Fraud Radar & Velocity Analysis

- **Description:** Analyzes vote velocity bursts (<2s), duplicate IP clusters, and double voting to output threat score (0-100%).
- **Route:** GET /api/v1/ai/fraud-analysis/:election\_id

## 5. Non-Functional Requirements

### 5.1 Performance

| Metric            | Requirement                                                        |
|-------------------|--------------------------------------------------------------------|
| API Response Time | < 2 seconds for standard data endpoints under normal load          |
| File Upload       | Handles CSV files up to 10 MB; uploads complete within 30 seconds  |
| Block Mining      | PoW block mined and synced to database within 2 seconds            |
| Concurrent Users  | Must support at least 100 concurrent users (student project scale) |

### 5.2 Security

| Requirement         | Detail                                                       |
|---------------------|--------------------------------------------------------------|
| Authentication      | JWT Bearer tokens, 24-hour expiry                            |
| Password Storage    | bcrypt with salt rounds = 10                                 |
| Token Blacklisting  | Logged-out tokens are blacklisted in memory                  |
| Rate Limiting       | Auth routes: strict limiter; Upload routes: upload limiter   |
| CORS                | Whitelist-only: localhost:3000 and the Vercel production URL |
| Input Validation    | Pydantic schemas for all incoming request data               |
| File Security       | MIME type validation; directory traversal prevention         |
| Vote Encryption     | AES-256 GCM symmetric vote payload encryption                |
| Transaction Signing | ECDSA SECP256R1 asymmetric digital signatures                |

### 5.3 Reliability

- System should maintain **99% uptime** during academic hours (dependent on Vercel and cloud SLAs)
- All database queries include indexed fields to prevent full collection scans

### 5.4 Usability

- Responsive design using Tailwind CSS — supports desktop, tablet, and mobile viewports
- Framer Motion animations for smooth page transitions and micro-interactions
- Consistent sidebar-based navigation for all dashboard roles

### 5.5 Scalability

- Relational database allows horizontal scaling if the student count grows
- Stateless JWT authentication design allows multiple backend instances

### 5.6 Maintainability

- Clear separation of concerns: routes -> services -> models -> blockchain engine

## 6. System Constraints

- 22. Blockchain PoW difficulty target is set to 2 leading zeros for lightweight student project execution.
- 23. File Uploads are stored on server local directory — not cloud storage (a limitation for scalability).
- 24. Token Blacklist is stored in memory, meaning it resets on server restart.
- 25. Vote attempts are limited to one per student per election, enforced by unique compound database constraints.
- 26. Admin Password Changes must be done by re-creating the account (no self-service for admins).
- 27. The system assumes stable internet connectivity for real-time WebSocket updates.

## 7. External Interface Requirements

### 7.1 User Interfaces

| Page                  | Role             | Description                                                            |
|-----------------------|------------------|------------------------------------------------------------------------|
| Home / Landing Page   | Public           | College landing page with login options                                |
| Admin Login           | Admin            | Separate login page for admin accounts                                 |
| Student/Faculty Login | Student, Faculty | Shared login page with role selection                                  |
| Admin Dashboard       | Admin            | Sidebar with Home, Elections, Candidates, Bulk Enrollment, Audit       |
| Student Dashboard     | Student          | Sidebar with Home, Elections, Candidates, Vote, Receipts, Verification |
| Faculty Dashboard     | Faculty          | Sidebar with Home, Live Results, Explorer, AI Fraud Radar, Settings    |
| Settings              | Student, Faculty | Password change form                                                   |

## 7.2 Hardware Interfaces

- No special hardware required; standard computer, tablet, or smartphone with internet access

## 7.3 Software Interfaces

| System              | Interface                                     |
|---------------------|-----------------------------------------------|
| SQLite / PostgreSQL | SQLAlchemy ORM connection protocol            |
| FastAPI WebSockets  | Real-time bidirectional event manager         |
| Vercel              | Static hosting via Vercel deployment pipeline |

## 7.4 Communication Interfaces

- **Protocol:** HTTPS (production), HTTP (local development)
- **Data Format:** JSON (REST API)
- **Auth Header:** Authorization: Bearer <JWT\_TOKEN>

# 8. Data Requirements

## 8.1 Data Entities (Degree College Specific)

| Entity      | Collection / Table | Key Fields                                                                         |
|-------------|--------------------|------------------------------------------------------------------------------------|
| Admin       | users              | fullName, email, password (hashed), role='admin'                                   |
| DC Student  | users              | fullName, email, password, rollNo, degree, year, semester, class, role='voter'     |
| DC Faculty  | users              | fullName, email, password, subject, course, role='observer'                        |
| Election    | elections          | title, description, status, start_time, end_time, created_by                       |
| Candidate   | candidates         | election_id, name, party, manifesto, avatar_url, vote_count                        |
| Vote        | votes              | user_id, election_id, candidate_id, voter_hash, tx_hash, block_index, receipt_hash |
| Block       | blocks             | index, timestamp, previous_hash, hash, nonce, merkle_root, signature               |
| Transaction | transactions       | tx_hash, block_index, election_id, voter_hash, encrypted_vote, signature           |
| Audit Log   | audit_logs         | user_id, user_email, action, details, ip_address, timestamp                        |

## 8.2 Data Retention

- Student audit and activity logs are timestamped and indexed; no automatic purging defined
- Blockchain blocks and vote transactions are immutable — once mined, records persist permanently

## 8.3 Data Integrity

- Unique email and username constraints on User collection
- Unique compound constraint on Vote (user\_id, election\_id) — one vote per student per election
- Unique index on Block (index, hash) and Transaction (tx\_hash) to guarantee ledger uniqueness

**UML**

# UML Diagrams — Complete Set

## TRCAC Blockchain Voting System — Degree College Module

### Table of Contents

1. Use Case Diagram
2. Class Diagram
3. Sequence Diagram — Login Flow
4. Sequence Diagram — Encrypted Vote & Block Mining
5. Sequence Diagram — Receipt Verification Flow
6. Activity Diagram — AI Fraud Detection & Velocity Marking
7. Component Diagram
8. ER Diagram (Database)

### 1. Use Case Diagram

Each role is shown in a separate diagram for A4 readability.

#### 1.1 Admin Use Cases

Admin -> [Login / Logout, Create Student Account, Create Faculty Account, Manage Elections, Manage Candidates, Bulk Enroll Students, Bulk Add Candidates, View Dashboard Stats, View & Delete Users, View Audit Logs]

## **1.2 DC Faculty / Observer Use Cases**

DC Faculty -> [Login / Logout, Change Password, View Dashboard, View Live Standings, View Block Explorer, Audit Chain Integrity, View AI Fraud Alerts, Download Reports]

## **1.3 DC Student / Voter Use Cases**

DC Student -> [Login / Logout, Change Password, View Dashboard, View Active Elections, View Candidate Profiles, Cast Encrypted Vote, Download QR Receipt, Verify Receipt Hash]

## 2. Class Diagram

Split into three focused groups for clarity on A4.

### 2.1 User & Auth Classes

**User Base Class:** +id, +fullName, +email, +password, +role, +lastLogin | +login()

**extends DCStudent:** +rollNo, +degree, +year, +semester, +class, +college | +changePassword()

**extends DCFaculty:** +subject, +courses[], +teachingAssignments[], +college | +changePassword()

### 2.2 Content & Progress Classes

**Election Class:** +id, +title, +description, +status, +startTime, +endTime

**Candidate Class:** +id, +electionId, +name, +party, +manifesto, +voteCount

**Block Class:** +index, +timestamp, +previousHash, +hash, +nonce, +merkleRoot, +signature | +calculateHash()

### 2.3 Assessment & Activity Classes

**Vote Class:** +id, +userId, +electionId, +candidateId, +voterHash, +encryptedVote, +txHash, +receiptHash

**AuditLog Class:** +id, +userId, +userEmail, +action, +details, +ipAddress, +timestamp

### 3. Sequence Diagram — Login Flow

Student -> React Frontend -> POST /api/v1/auth/login -> FastAPI -> Search User by email -> bcrypt verify password -> Generate JWT Token (24h) -> Return 200 {token, user} -> Save to storage -> Redirect to Dashboard.

### 4. Sequence Diagram — Encrypted Vote & Block Mining

Student -> Select Candidate -> Click Vote -> AES-256 GCM Encrypt -> ECDSA Sign -> POST /api/v1/votes/cast -> FastAPI -> Check single-vote rule -> Queue Transaction -> Mine PoW Block (calculate Merkle Root) -> Save Vote to DB -> Broadcast WebSockets -> Return QR Receipt.

### 5. Sequence Diagram — Receipt Verification Flow

Student / Auditor -> Enter Receipt Hash -> GET /api/v1/votes/verify-receipt/:hash -> FastAPI -> Lookup Vote by receipt\_hash -> Search Blockchain Ledger for tx\_hash -> Verify block index -> Return {verified: True, block\_index, tx\_hash, voter\_hash}.

## 6. Activity Diagram — AI Fraud Detection & Velocity Marking

Start -> Vote Submitted -> Read Timestamp & Client IP -> Compute Delta Time between consecutive votes -> Delta < 2.0s? -> Yes: Flag High Velocity Anomaly -> Count IP submissions > 5? -> Yes: Flag IP Concentration -> Double-Vote attempts detected? -> Yes: Add Double-Vote Penalty -> Calculate Unified Fraud Risk Score (0-100%) -> Output Risk Level (Low/Medium/High/Critical) -> End.

## 7. Component Diagram

7.1 Frontend Components (React SPA: AuthContext, Sidebar, AdminDashboard, VoterDashboard, ObserverDashboard, Explorer)

7.2 Backend Components (FastAPI Router: auth.py, elections.py, votes.py, blockchain.py, admin.py, ai.py, observer.py)

7.3 Full System Overview (Browser -> HTTPS/WSS -> FastAPI ASGI Server -> Blockchain Engine + SQLite/PostgreSQL Database)

## 8. Entity-Relationship (ER) Diagram

8.1 Core User & Content Entities: users (1:N) elections, elections (1:N) candidates

8.2 Voting & Ledger Entities: users (1:N) votes, elections (1:N) votes, blocks (1:N) transactions

**ADD**

# Architecture Design Document (ADD)

## TRCAC Blockchain Voting System — Degree College Module

Version: 1.0 | Date: July 2026 | Scope: Degree College Only

### 1. Introduction & Goals

This document describes the structural design of the TRCAC BVS Degree College Module — how it is organised, why key decisions were made, and how all parts connect.

#### Architectural Goals

| Goal                   | Description                                                                         |
|------------------------|-------------------------------------------------------------------------------------|
| Separation of Concerns | UI never touches database directly — frontend and backend are fully separate        |
| Role-Based Security    | Every protected route enforces Admin / Faculty / Student role checks                |
| Stateless Auth         | JWT-based authentication allows multiple backend instances without shared sessions  |
| Simplicity             | Clean architecture — Python 3.12 FastAPI backend + React 19 frontend                |
| Maintainability        | Modular clean code, environment variables, and clear directory structure throughout |

## 2. System Context

### 2.1 Actors and External Systems

| Actor / System      | Type             | Role                                                        |
|---------------------|------------------|-------------------------------------------------------------|
| Admin               | User             | Creates accounts, manages elections/candidates, bulk ops    |
| DC Faculty          | User             | Monitors live results, audits ledger, checks AI fraud radar |
| DC Student          | User             | Casts encrypted votes, generates receipts, verifies hashes  |
| SQLite / PostgreSQL | External Service | Cloud-hosted / local relational database                    |
| Custom Blockchain   | Core Engine      | Hosts and mines immutable SHA-256 block ledger              |
| Vercel              | External Service | Hosts and serves the React frontend (CDN)                   |

## 3. Overall Architecture — Three-Tier Model

### 3.1 Architecture Layers

- **Tier 1 — Presentation:** Browser -> React + Vite + Tailwind CSS Single Page Application
- **Tier 2 — Application:** FastAPI REST API (Auth Middleware, AI Fraud Radar, WebSockets)
- **Tier 3 — Data & Ledger:** Custom Python SHA-256 Blockchain Engine + SQLite/PostgreSQL Database

## 4. Frontend Architecture

### 4.1 Technology Stack

| Technology       | Version   | Role                              |
|------------------|-----------|-----------------------------------|
| React            | 19.0      | UI library                        |
| Vite             | 5.x / 7.x | Build tool + HMR dev server       |
| React Router DOM | 6.x       | Client-side routing               |
| Tailwind CSS     | 3.4       | Utility-first styling & Dark Mode |
| Framer Motion    | 11.x      | Page transitions and animations   |
| Axios            | 1.x       | HTTP client for API calls         |
| Recharts         | 2.x       | Live vote distribution charts     |

### 4.2 Routing Structure

- /login -> Shared login page with role detection
- /admin -> Protected Admin Dashboard (Overview, Elections, Candidates, Import)
- /voter -> Protected Voter Dashboard (Active Ballots, Cast Vote, QR Receipt)
- /observer -> Protected Faculty / Observer Live Standings
- /explorer -> Public Blockchain Ledger Browser

## 5. Backend Architecture

### 5.1 Technology Stack

| Technology   | Version | Role                                     |
|--------------|---------|------------------------------------------|
| Python       | 3.12+   | Server runtime                           |
| FastAPI      | 0.111+  | Web framework                            |
| SQLAlchemy   | 2.0+    | Database ORM                             |
| PyJWT        | 2.8+    | JWT auth tokens                          |
| bcrypt       | 4.1+    | Password hashing                         |
| Cryptography | 42.0+   | ECDSA SECP256R1 & AES-256 GCM algorithms |
| WebSockets   | 12.0+   | Real-time push event manager             |

### 5.3 Rate Limiting Strategy

| Route Group   | Limiter Type          | Reason                               |
|---------------|-----------------------|--------------------------------------|
| /api/v1/auth  | Strict (auth limiter) | Block brute-force login              |
| /api/v1/votes | Vote limiter          | Prevent automated double-vote bursts |
| /api/v1/admin | Upload limiter        | Prevent file upload abuse            |
| All others    | General limiter       | General DDoS protection              |

### 5.4 API Route Map

| Prefix             | File          | Key Endpoints                          |
|--------------------|---------------|----------------------------------------|
| /api/v1/auth       | auth.py       | login, register, me                    |
| /api/v1/elections  | elections.py  | create, list, status, add candidate    |
| /api/v1/votes      | votes.py      | cast, my-receipts, verify-receipt      |
| /api/v1/blockchain | blockchain.py | blocks, search, status, verify-chain   |
| /api/v1/admin      | admin.py      | metrics, import-voters-csv, audit-logs |
| /api/v1/ai         | ai.py         | fraud-analysis                         |
| /api/v1/observer   | observer.py   | live-results                           |

## 6. Database Architecture

### 6.1 Collections / Tables Summary (DC Scope)

| Collection / Table | Model            | Purpose                               |
|--------------------|------------------|---------------------------------------|
| users              | User             | Admin, student, and faculty accounts  |
| elections          | Election         | Election metadata and lifecycle state |
| candidates         | Candidate        | Candidate nominations and tallies     |
| votes              | Vote             | Encrypted vote records and receipts   |
| blocks             | BlockModel       | Mined blockchain block headers        |
| transactions       | TransactionModel | Encrypted ledger transactions         |
| audit_logs         | AuditLog         | Immutable security action logs        |

### 6.2 Key Indexes

| Collection / Table | Index                  | Type            |
|--------------------|------------------------|-----------------|
| users              | email, username        | Unique          |
| votes              | (user_id, election_id) | Unique Compound |
| votes              | tx_hash, receipt_hash  | Unique          |
| blocks             | index, hash            | Unique          |
| transactions       | tx_hash                | Unique          |
| audit_logs         | (user_id, timestamp)   | Compound        |

### 6.3 Degree College Class Name Convention

| Degree    | Code  | FY      | SY      | TY      |
|-----------|-------|---------|---------|---------|
| B.Sc (CS) | BScCS | FYBScCS | SYBScCS | TYBScCS |
| B.Sc (IT) | BScIT | FYBScIT | SYBScIT | TYBScIT |
| BA        | BA    | FYBA    | SYBA    | TYBA    |
| BMS       | BMS   | FYBMS   | SYBMS   | TYBMS   |
| BCom      | BCom  | FYBCom  | SYBCom  | TYBCom  |
| BAF       | BAF   | FYBAF   | SYBAF   | TYBAF   |
| BAMMC     | BAMMC | FYBAMMC | SYBAMMC | TYBAMMC |

## 7. Authentication & Authorization Architecture

### 7.1 JWT Token Structure

| Field        | Value                                       |
|--------------|---------------------------------------------|
| Algorithm    | HS256                                       |
| sub (userId) | Database integer ID of user                 |
| role         | student / faculty / admin                   |
| email        | User registered institutional email address |
| exp          | Issued at + 24 hours                        |

## 9. Security Architecture

### 9.2 Security Measures Summary

| Threat                   | Countermeasure                       | Implementation                                  |
|--------------------------|--------------------------------------|-------------------------------------------------|
| Brute force login        | Strict rate limiter                  | Rate limiting middleware on /api/v1/auth        |
| Token reuse after logout | Token blacklist / localStorage wipe  | Client-side storage wipe + server blacklist     |
| Password cracking        | bcrypt hashing                       | 10 salt rounds with salted hashes               |
| Cross-origin requests    | CORS whitelist                       | Only frontend origin URL allowed                |
| Clickjacking             | X-Frame-Options: DENY                | FastAPI security headers middleware             |
| Vote choice exposure     | AES-256 GCM encryption               | Vote payload encrypted with 32-byte secret key  |
| Transaction forgery      | ECDSA SECP256R1 signatures           | Digital signatures verified before block mining |
| SQL Injection            | SQLAlchemy ORM parameterized queries | Zero raw string SQL concatenation               |
| Secret leakage           | dotenv + .gitignore                  | SECRET_KEY, DATABASE_URL in .env file           |

## 10. Deployment Architecture

### 10.2 Environment Variables

| Variable              | Used In | Purpose                                           |
|-----------------------|---------|---------------------------------------------------|
| DATABASE_URL          | Backend | SQLite / PostgreSQL database connection string    |
| SECRET_KEY            | Backend | JWT signing secret key                            |
| VOTE_ENCRYPTION_KEY   | Backend | 32-byte AES-256 GCM vote payload key              |
| BLOCKCHAIN_DIFFICULTY | Backend | Proof-of-Work leading zero difficulty (default 2) |
| PORT                  | Backend | Server port (default 8000)                        |

## 11. Key Design Decisions

| # | Decision     | Choice                        | Rationale                                           | Tradeoff                                  |
|---|--------------|-------------------------------|-----------------------------------------------------|-------------------------------------------|
| 1 | User storage | Unified users table with role | Unified auth lookup across admin, students, faculty | Role check required on protected routes   |
| 2 | Vote Secrecy | AES-256 GCM encryption        | Prevents ballot choice from being public on ledger  | Server holds symmetric key for decryption |
| 3 | Voter Auth   | ECDSA SECP256R1 signatures    | Digital signatures guarantee vote authenticity      | Keypair generated per vote transaction    |
| 4 | Real-Time    | FastAPI WebSockets            | Instant push broadcasts to live dashboards          | Active socket connection management       |
| 5 | Threat Radar | Heuristic Velocity Analysis   | Detects bursts (<2s), duplicate IPs, double voting  | Thresholds require tuning for scale       |
| 6 | Tech stack   | Python FastAPI + React 19     | High performance, asynchronous, modern ecosystem    | Two language environments (Python + TS)   |